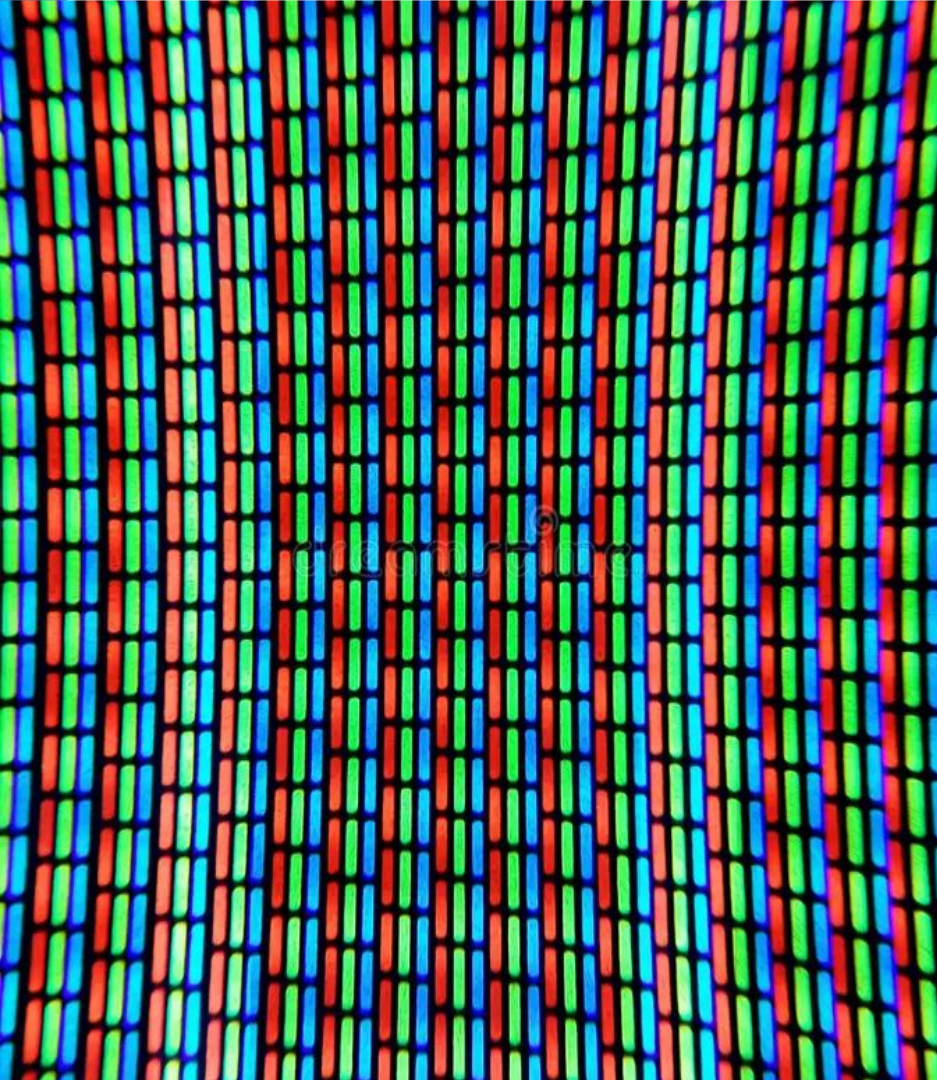


Colour Spaces

RGB . HSV . YCbCR . LAB

Mia | Romey | Kyra | Rebecca | Julia

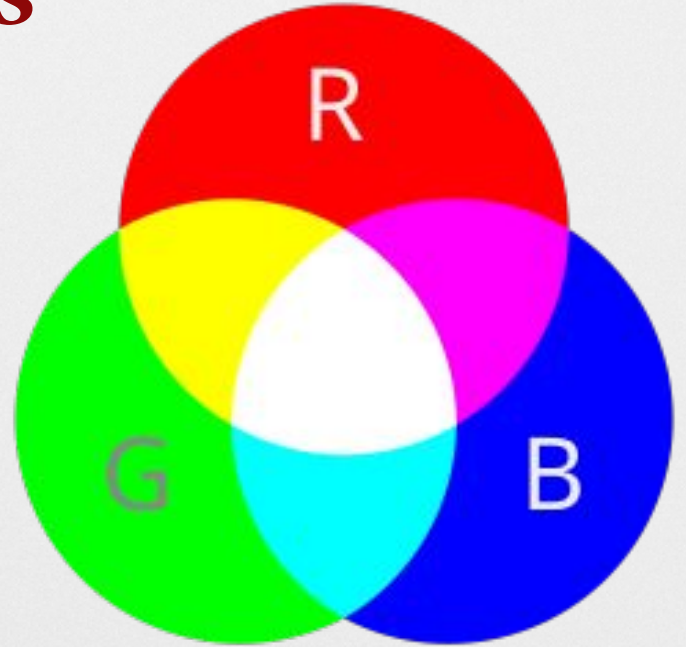


01

RGB

RGB Fundamentals

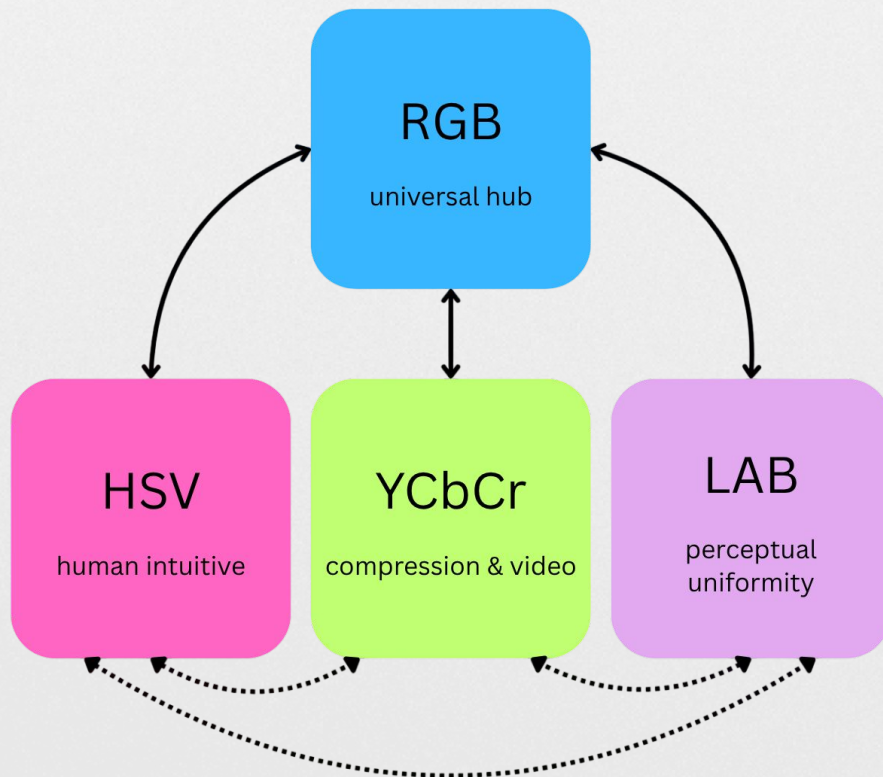
- Represents colors as a combination of Red, Green and Blue
- Works well in image editing
 - provides almost the true color of the image

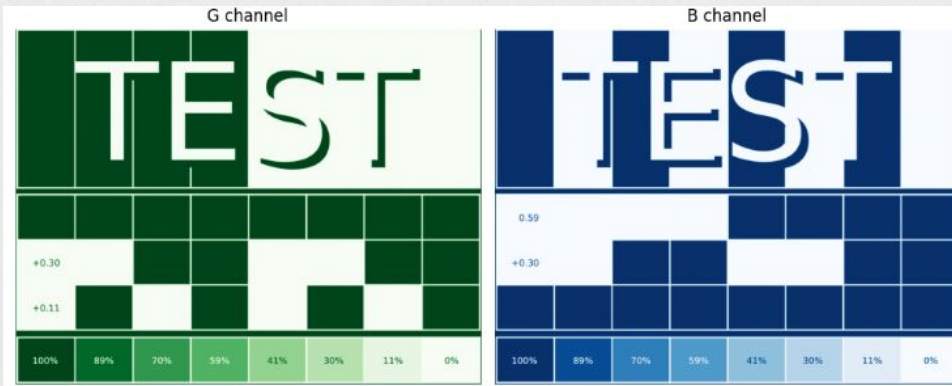
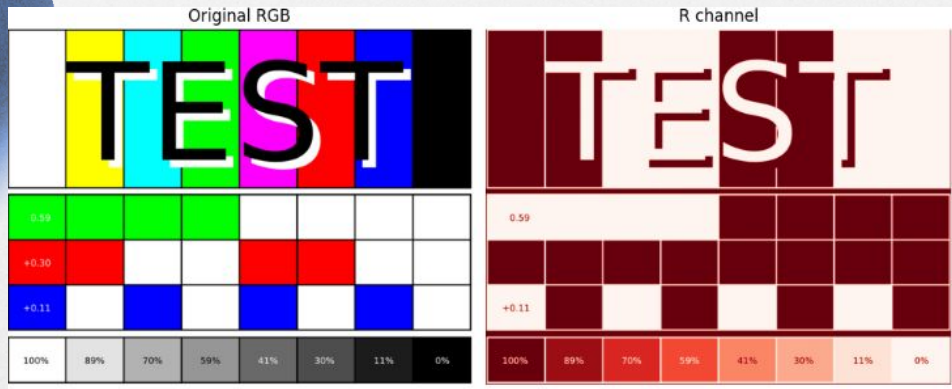


RGB Applications

Applications	Alternatives
1. Digital Archiving and Conservation 2. Web design 3. Professional Image Editing	1. Video/streaming → YCbCr 2. Colour detection → HSV 3. Print/colour accuracy → LAB

RGB in Conversions





Failure cases & limitations

- Red, Green and Blue channels are highly dependant on each other
- Inefficient for data compression
 - The channels contain redundant information that wastes bandwidth

02

HSV

Original



H (hue)



S (saturation)

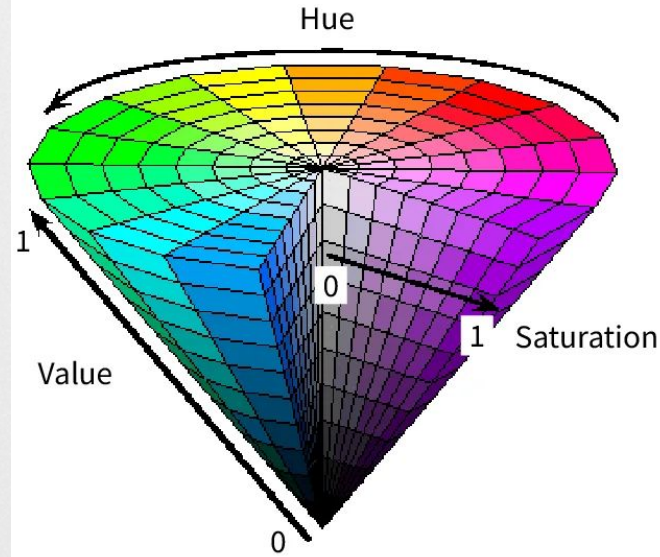
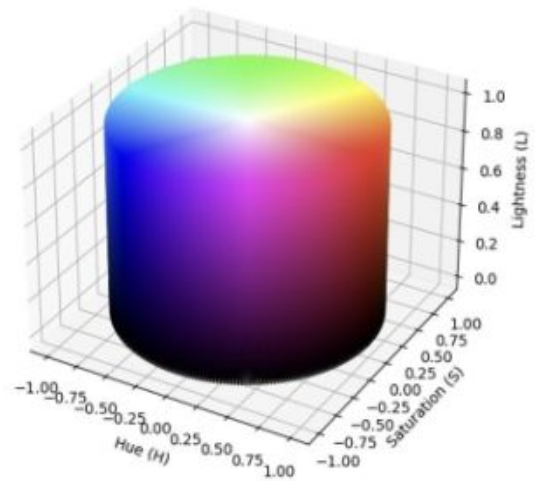


V (brightness)



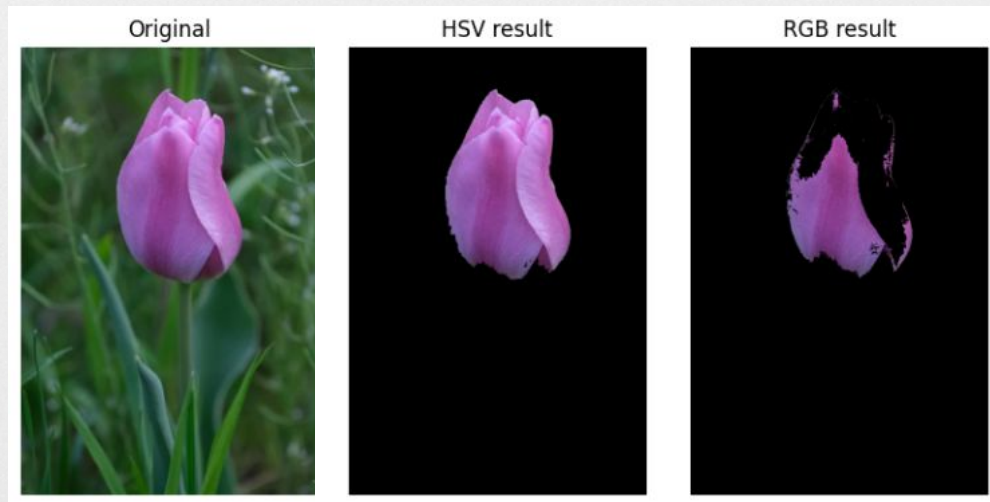
HSV Fundamentals

- 3 human-intuitive axes: **Hue** (what colour), **Saturation** (how vivid), and **Value** (how bright)
- Designed by Alvy Ray Smith in 1978 so artists could think "bright saturated blue" instead of hunting through RGB numbers
- Changing brightness only affects V, leaving the colour itself untouched



Real World Application: Colour Masking

- HSV is the standard colour space for object detection by colour in computer vision
- A hue range threshold isolates a specific colour regardless of lighting. This is something RGB cannot do reliably
- Used in traffic light detection, fire detection, and medical imaging



Same tulip, same threshold. HSV isolates cleanly, RGB fragments under shadow.

RGB $\rightarrow$ HSV

Step 1: Normalise R, G, B to [0, 1]

Step 2: $V = \max(R, G, B)$

Step 3: $S = \delta / V$

where $\delta = \max - \min$ ($S = 0$ if $V = 0$)

Step 4: $H = 60^\circ \times$ formula based on which channel is dominant

Refer to the worked out example:

Input colour (RGB 0-255)

R

255

G

140

B

0

Target space

Convert to

HSV

#FABC00

Why HSV exists: RGB encodes colour and brightness together. HSV reorganises them into a cylinder matching human intuition: which colour (hue), how vivid (saturation), how bright (value). To isolate red objects, threshold H alone — not a mix of R, G, and B.

Step-by-step: RGB $\rightarrow$ HSV

Step	Formula	Value
Inputs	R, G, B	0.980, 0.549, 0.000
V (value)	$V = \max(R, G, B)$	0.9804
Cmin	$C_{\min} = \min(R, G, B)$	0.0000
C (chroma)	$C = V - C_{\min}$	0.9804
S (sat.)	$S = C / V$ (0 if $V=0$)	1.0000
Dominant	Which primary = V?	R
H (hue)	$H = 60 \times$ sector formula (degrees)	33.60°

Result:

H = 33.60°

S = 1.0000

V = 0.9804

HSV Limitations

Situation	Problem	Use instead
Measuring how different two colours look	HSV distance $\neq$ perceptual difference	LAB
Compression and video transmission	HSV not designed for subsampling	YCbCr
Objects that are grey, white, or pastel	Hue is unreliable at low saturation	OpenCV fix – S min threshold

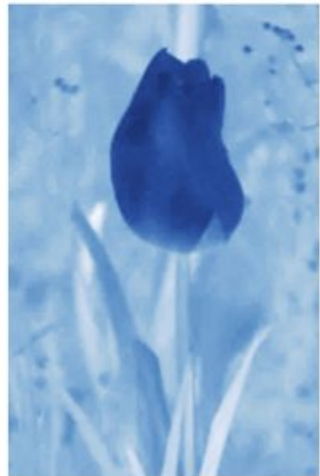
Original RGB



Y (Luma)



Cb (Blue diff)



Cr (Red diff)



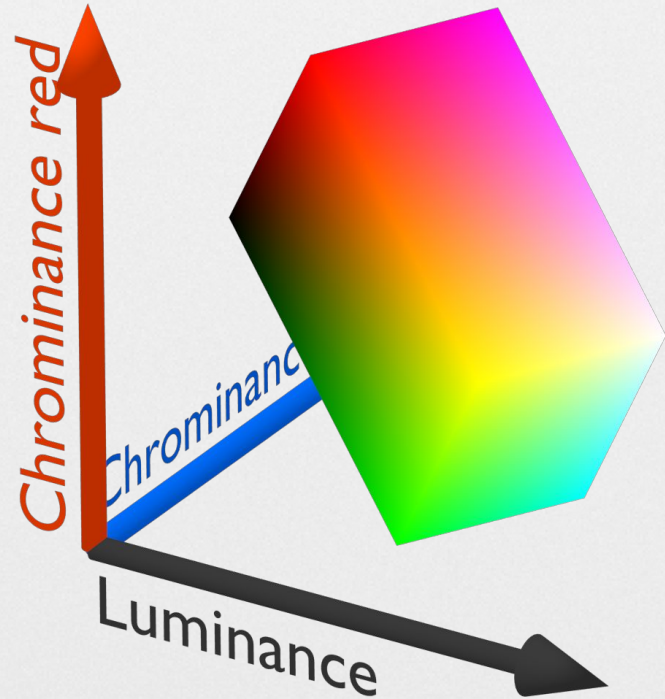
03

YCbCr

YCbCr

Fundamentals

- Colour space used in digital image and video compression
- Separates luminance (Y') from chrominance (Cb, Cr)
- Y' stores image detail, Cb and Cr store colour information
- Enables efficient compression with minimal visible quality loss



RGB $\leftrightarrow$ YCbCr Conversion

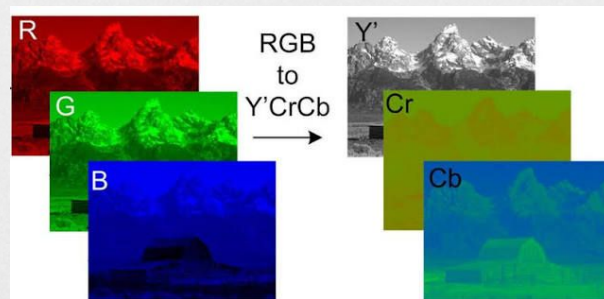
- Images are usually stored in RGB and then converted to YCbCr before compression
- Conversion separates luminance (Y) from chrominance (Cb, Cr)
- Based on weighted combinations of RGB values (BT.601 standard)

$$Y = 0.299R + 0.587G + 0.114B + 16$$

$$Cb = -0.1687R - 0.3313G + 0.500B + 128$$

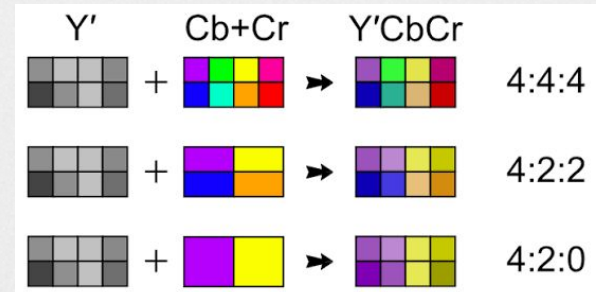
$$Cr = 0.500R - 0.4187G - 0.0813B + 128$$

- Enables efficient chroma subsampling and compression



Chroma Subsampling

- Reduces colour resolution to save storage and bandwidth
- Human vision is less sensitive to colour details than brightness detail
- Formats:
 - 4:4:4 - No subsampling
 - 4:2:2 - Half horizontal colour resolution
 - 4:2:0 - Half horizontal and vertical colour resolution

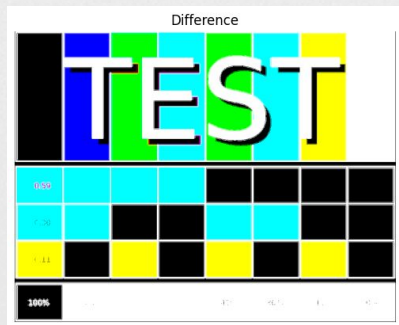
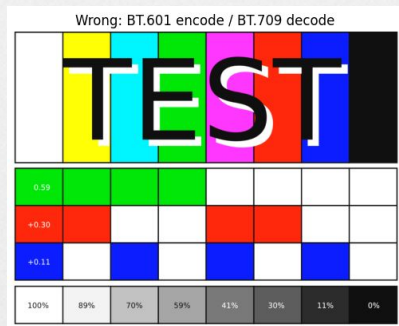


Skin Detection

- Skin tones cluster in a narrow range of Cb and Cr values
- Y (brightness) is ignored → reduces lighting effects
- Enables simple threshold-based segmentation
- Works across different skin tones and lighting conditions



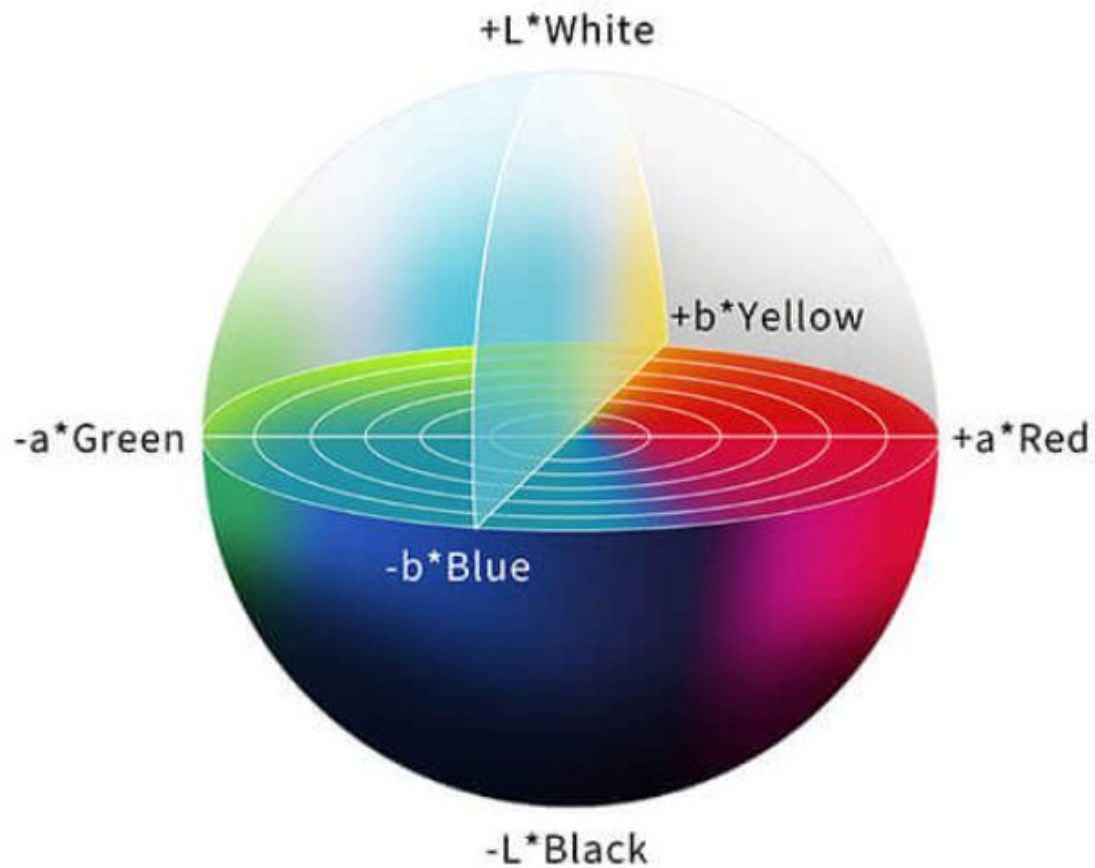
Failure cases & limitations



- Chroma subsampling may cause colour bleeding and compression artifacts
- Fine details such as text and UI elements can appear blurred
- Not ideal for professional graphics or colour-critical editing
- RGB is preferred for colour-critical applications
- Excessive compression reduces image quality visibly

04

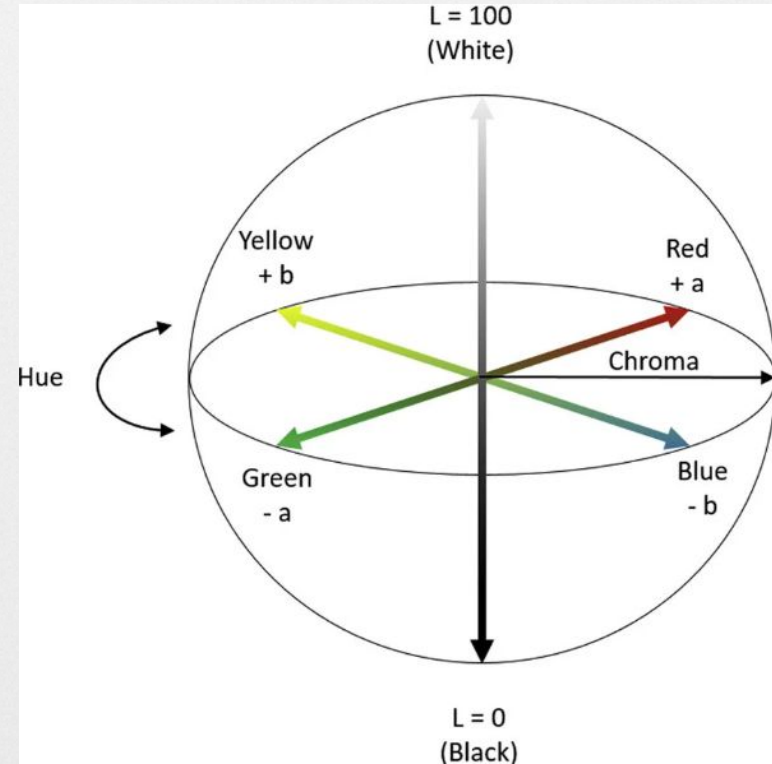
LAB



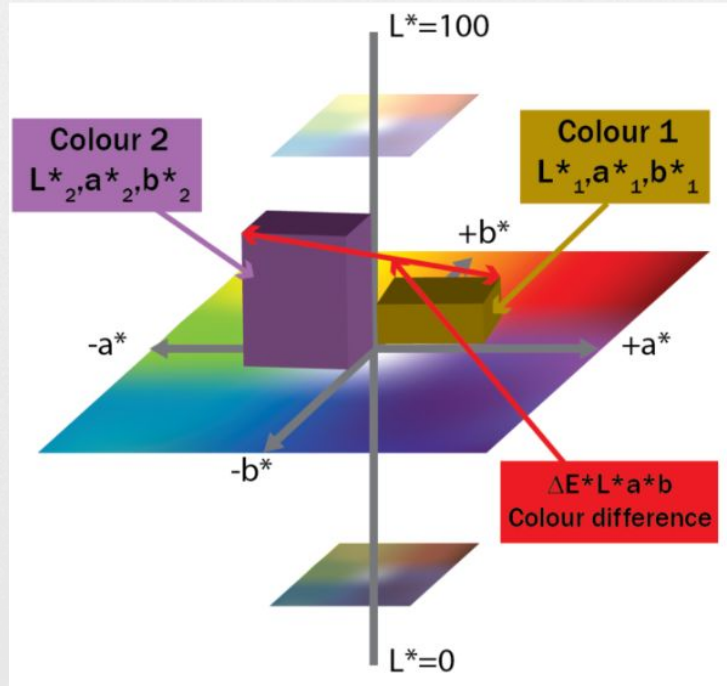
LAB fundamentals

We have 3 LAB axes:

- L^* - lightness
 - a^* - green-red
 - b^* - blue-yellow
-
- Designed to model human colour perception, separated brightness from colour information.
 - The separation makes LAB useful in image processing because lighting and colour can be analyzed independently



ΔE^* - Measuring Colour Difference in LAB



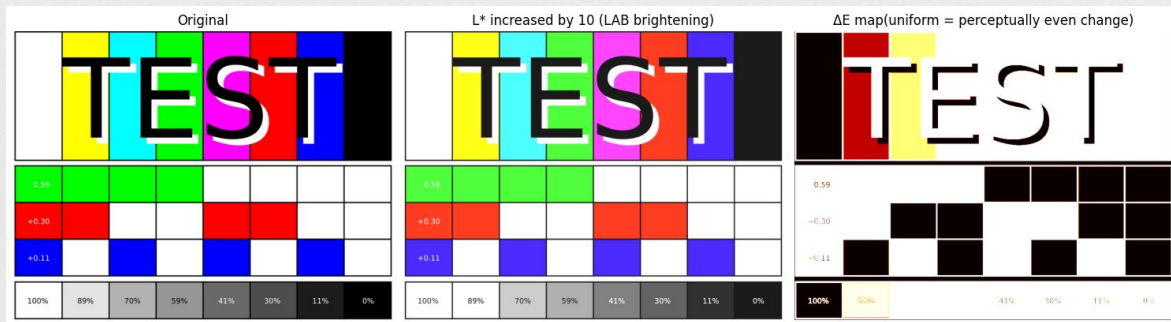
$$\Delta E^* = \sqrt{[(\Delta L^*)^2 + (\Delta a^*)^2 + (\Delta b^*)^2]}$$

- Perceptually uniform
 - $\Delta E^* = 1$ - approx the smallest difference eyes can detect
- $\Delta E = 0$ Colours are identical
- Small ΔE Colours are very similar
 - Large ΔE Colours are noticeably different
- < 1 Not visible to the human eye
- $1 - 2$ Only noticeable by a trained expert $2 - 3.5$ Acceptable in most industries
- > 5 Obvious mismatch (rejected)

Failure cases & limitations

- Not perfectly perceptually uniform, highly saturated colours can still produce inaccurate ΔE^* values
- Depends on accurate RGB calibration- poor lighting or camera calibration makes LAB values unreliable
- Computationally expensive, the 5-step pipeline is significantly slower than working directly in RGB or HSV
- Noise amplification ,dark regions become noisier after conversion
- Hue instability near neutral when a^* and b^* are close to 0, small rounding errors cause large hue shifts

These limitations led to improved formulas -CIE94 and CIEDE2000, developed to better model human perception



Normalise

Linearise

XYZ Matrix

White Point

$f(t)$

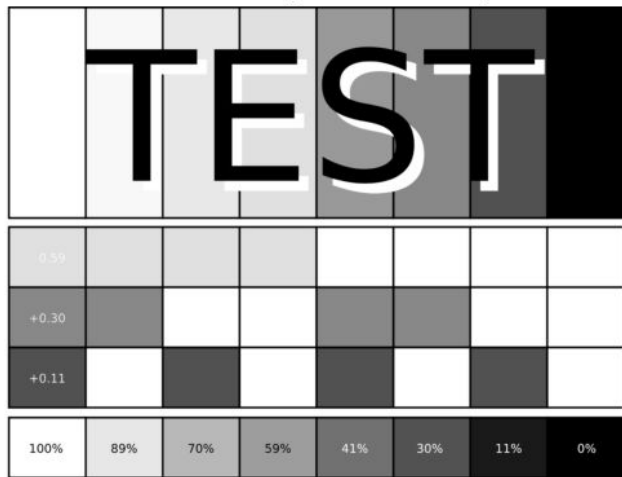
LAB

RGB $\rightarrow$ LAB conversion

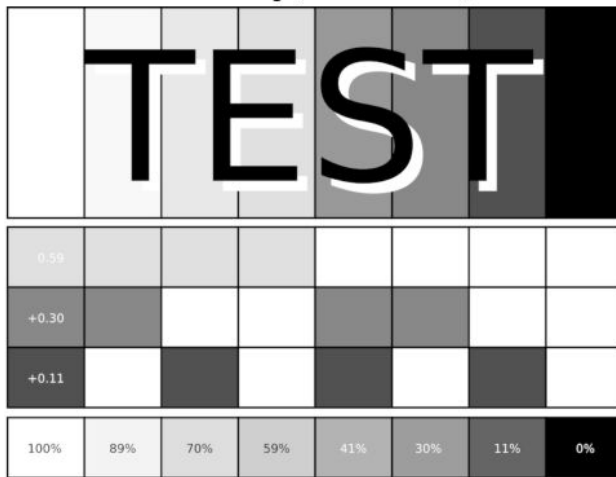
- RGB must be converted in two stages - first to XYZ, then to LAB
- XYZ acts as a neutral middle ground that any colour system can connect to
- 5 Step Pipeline
- LAB is the destination - equal steps in LAB look equally different to the human eye

Failure Case: Skipping Linearisation for LAB

L* — Correct (with linearisation)



L* — Wrong (no linearisation)



Difference



Conversion Limitations

- When speed is critical
- Working within entirely one space
- When OpenCV already handles it



Thank you for your
attention !